

PRODUCTION READINESS AUDIT · THIRD PASS · V3

NovaFacts

Production Readiness Audit Report

Date	2026-06-26
Prior pass	v2 — 2026-06-25 · Score 18 / 100
Auditor	Senior Software Architect Review (AI-assisted)
Scope	project-backend/ · project-frontend/frontend/ · project-docs/
Method	All findings verified by reading the cited file at the stated line
Stack	Spring Boot 3.5 · Java 21 · Vue 3 · PostgreSQL · JWT 0.12.6

34
/ 100

Not Production Ready

JWT infrastructure is now complete and endpoints are protected. The system issues real tokens, stores them, and attaches them to requests. One missing `router.push()` call is the only gap between the current state and a working login flow. The core domain remains unimplemented.

▲ +16 pts vs v2

1. Executive Summary

Significant progress — authentication infrastructure is now real

JWT tokens are generated, signed with a configurable secret, returned in `LoginResponse`, stored in `localStorage`, and attached to every subsequent request via an Axios interceptor. The `/api/users/**` endpoints are now gated behind authentication. Database credentials are fully externalized. `httpBasic()` is removed. The JPA entity has been corrected to map to the `usuario` table with accurate column bindings. These are the largest improvements since the project began.

Login is still non-functional from the user's perspective

`LoginView.vue:127-128` receives a valid JWT, stores it, then stays on `/`. `useRouter()` is not imported. The user remains on the login page regardless of whether authentication succeeded. One `router.push('/dashboard')` call closes this gap entirely. Additionally, three medium-severity regressions were introduced during this cycle.

Score by Dimension

Dimension	v2	v3	Δ	Key driver
Security	10	38	+28	JWT, endpoint protection, creds out, httpBasic gone
Authentication	5	55	+50	JWT issued, stored, sent on all requests
Authorization	0	20	+20	<code>/api/users/**</code> now requires a valid token
Completeness	10	15	+5	JWT integration nearly complete; core domain absent
Backend Architecture	45	52	+7	JPA mapping fixed; no validation/exception handler
Frontend Architecture	30	33	+3	<code>api.ts</code> abstraction; dual interceptor regression
Infrastructure	35	45	+10	Secrets externalized, README, Docker port correct
Testing	40	40	0	No change
Observability	10	10	0	No change
Documentation	40	50	+10	Backend <code>README.md</code> added

2. Status of Prior Findings

2.1 Critical Severity

C-01 CRITICAL PARTIAL No real authentication issued — login is a functional dead end

Category Security / Authentication

Files `auth/service/UserService.java:68` · `src/views/LoginView.vue:127`

What was implemented (this cycle): The entire JWT backend infrastructure is now present and correct. `JwtService` generates tokens via JJWT 0.12.6 signed with `${JWT_SECRET}` (no hardcoded fallback). `UserService.java:68` calls `jwtService.generateToken()` and returns the token in `LoginResponse`. `JwtAuthenticationFilter` validates tokens on every request and populates the `SecurityContext`. On the frontend, `authService.ts:22-28` reads `response.data.token` and stores it in `localStorage`. The `api.ts` interceptor attaches a `Bearer` header to every subsequent request.

What blocks resolution: `LoginView.vue:127-128`:

```
console.log('Login exitoso para el token:', result.token);  
// Aquí puedes redirigir con tu vue-router ej: router.push('/dashboard')
```

`useRouter()` is never imported. The user stays on `/` regardless of outcome. The JWT is issued, stored, and will be sent on all future requests — one `router.push()` is the only gap.

RESOLUTION

Import `useRouter` from `vue-router`. Replace the `console.log` line with `router.push('/dashboard')`. Resolves this finding entirely.

C-02 CRITICAL ✓ RESOLVED Unauthenticated user enumeration and arbitrary account deletion

Files `config/SecurityConfig.java:37`

`SecurityConfig.java:37`: `.requestMatchers("/api/users/**").authenticated()` — JWT filter validates the token before the request reaches `UserController`. An unauthenticated `GET /api/users` or `DELETE /api/users/{id}` returns `401 Unauthorized` via `HttpStatusEntryPoint(HttpStatus.UNAUTHORIZED)` at line 41.

C-03

CRITICAL

✓ RESOLVED

Hardcoded database credentials committed to version control

Files

`application.properties:5` · `docker-compose.yml:11` · `.env.example` · `.gitignore`

`application.properties:5` : `spring.datasource.password=${POSTGRES_PASSWORD}` — no literal value. `docker-compose.yml` : `POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}` . JWT secret follows the same pattern (`${JWT_SECRET}` with no fallback — startup fails if unset). `.env.example` committed with documented placeholders. `.env` listed in `.gitignore` and never committed. `setup.sh` fails fast with an actionable error message if `.env` , `POSTGRES_PASSWORD` , or `JWT_SECRET` is absent.

C-04

CRITICAL

OPEN

Core business domain has no API, entity, or persistence layer

Files

`booking/model/Booking.java` · `booking/service/BookingValidator.java` · `booking/service/InvoiceCalculator.java`

All booking classes remain plain Java objects with no Spring annotations and no JPA mapping.

`Booking.java` has `private final double pricePerNight` with no `@Entity` . `booking/` has no `entity/` , `repository/` , or `controller/` subdirectory. None of the 14 domain tables in `Esquema_BD.sql` — `reserva` , `factura` , `anticipo` , `penalidad` , `nota_credito` , `devolucion` , `log_transaccion` , `propiedad` , `canal` , `temporada` , `politica_cancelacion` , `regla_penalidad` , `rol` — have a corresponding JPA entity. The system's stated business purpose is entirely absent from the API layer.

RESOLUTION

Implement domain entities and repositories following the `auth/` feature-package pattern and Spanish column-naming convention from `Esquema_BD.sql` . Add Flyway first (M-03) before introducing new entities.

2.2 High Severity

H-01 HIGH PARTIAL Login error messages leak whether a username exists

Files `auth/service/UserService.java:57,65`

The user-not-found case was improved: `UserService.java:57` now throws `ResponseStatusException(HttpStatus.UNAUTHORIZED, "Credenciales inválidas")` → 401 with a generic message.

The wrong-password case was **not** updated: `UserService.java:65` still throws `RuntimeException("Contraseña incorrecta")` → 500 with a distinct message.

A client can determine account existence: 401 + "Credenciales inválidas" means unknown user; 500 + "Contraseña incorrecta" means known user with wrong password. The enumeration vector is narrowed but not closed.

RESOLUTION

Change line 65 to `throw new ResponseStatusException(HttpStatus.UNAUTHORIZED, "Credenciales inválidas")`. Both failure paths then produce identical 401 responses.

H-02 HIGH OPEN Dead configuration classes outside Spring component-scan boundary

Files `com/novafacts/SecurityConfig.java` ·
`com/novafacts/config/PasswordConfig.java`

Both files still exist. Both carry `@Configuration` annotations in packages outside the component-scan root (`com.novafacts` vs. `com.novafacts.backend`). The `PasswordConfig.PasswordEncoder` bean, if accidentally activated, would cause a bean-ambiguity conflict with the live `SecurityConfig.passwordEncoder()`. The live `SecurityConfig` in the same-named class creates a naming confusion that will mislead every future developer reading the codebase.

RESOLUTION

Delete both files. No behavior changes — they are never loaded.

H-03 HIGH OPEN No global exception handler — domain errors surface as 500s

Files `auth/service/UserService.java:65` · no `@ControllerAdvice` anywhere

No `@ControllerAdvice` or `@ExceptionHandler` class exists in any package under `src/main/java`. The wrong-password path throws raw `RuntimeException` → 500. Any `DataIntegrityViolationException` (duplicate email, missing FK) is also unhandled → 500 with a Hibernate stack trace. `IllegalArgumentException` from `BookingValidator`, if ever wired to a controller, would also produce 500.

RESOLUTION

Add a `@ControllerAdvice GlobalExceptionHandler` in `com.novafacts.backend.common`. Map: `ResponseStatusException` → pass-through, `DataIntegrityViolationException` → 409, `IllegalArgumentException` → 400, all others → 500 with no internal detail.

H-04 HIGH OPEN No request validation anywhere in the API

Files `pom.xml` · `auth/dto/CreateUserRequest.java` · `auth/dto/LoginRequest.java`

`pom.xml` contains no `spring-boot-starter-validation` dependency. No DTO has a `jakarta.validation` annotation. No controller method has `@Valid`. `POST /api/users` with `{"username":"","password":""}` succeeds — `passwordEncoder.encode("")` produces a valid BCrypt hash of an empty string and the record is persisted.

RESOLUTION

Add `spring-boot-starter-validation` to `pom.xml`. Annotate DTOs with `@NotBlank`, `@Email`, `@Size`. Add `@Valid` to controller method parameters.

H-05 HIGH OPEN LoginView.vue completes login but performs no navigation

Files `src/views/LoginView.vue:127-128`

`handleFormSubmit()` handles error cases correctly but on `status === 'success'` executes only a `console.log`. The redirect is left as a comment. `useRouter()` is not imported. The user remains on the login page regardless of whether authentication succeeded. The JWT is now correctly issued and stored — this is the only line of code between the current state and a working login experience.

RESOLUTION

Import `useRouter`. On success, call `router.push('/dashboard')` and remove the `console.log` (which leaks the JWT to the browser console — see N-01 in new findings).

H-06 **HIGH** **OPEN** **SecretView.vue, SecretDisplay.vue, and /secret route are orphaned dead code**

Files `src/views/SecretView.vue` · `src/components/SecretDisplay.vue` · `src/router/index.js:14-17`

All three still present. Since `LoginView.vue` never calls `router.push()`, no code path navigates to `/secret`. The route parameter vs. query parameter mismatch (`:secretPhrase` param vs. `route.query.phrase` read in the component) is also still present — the component was never exercised end-to-end.

RESOLUTION

Delete `SecretDisplay.vue`, `SecretView.vue`, and remove the `/secret` route from `router/index.js`. Zero risk of regression.

H-07 **HIGH** **✓ RESOLVED** **HTTP Basic auth enabled alongside permitAll()**

Files `config/SecurityConfig.java`

`SecurityConfig.java` contains no `.httpBasic()` call. `SessionCreationPolicy.STATELESS` is configured at line 34. The authentication entry point is now `HttpStatusEntryPoint(HttpStatus.UNAUTHORIZED)` — unauthenticated requests receive a clean JSON 401, not a browser Basic-auth prompt.

H-08 **HIGH** **PARTIAL** **Username field has no uniqueness constraint**

Files `auth/entity/User.java:15`

`User.java:15`: `@Column(name = "email", nullable = false, unique = true, length = 150)` — the uniqueness constraint is now declared in the JPA entity and aligns with the existing `usuario_email_key` constraint in PostgreSQL (confirmed via `pg_constraint` query).

The service-level pre-check (409 Conflict before the DB constraint fires) is still absent. A duplicate-email insert produces an unhandled `DataIntegrityViolationException` → 500 (blocked by H-03).

RESOLUTION

Add a `userRepository.existsByUsername(email)` check in `createUser()` before saving. Throw `ResponseStatusException(HttpStatus.CONFLICT, "El correo ya está registrado")`. Requires H-03 to surface as a proper 409.

2.3 Medium Severity

ID	Title	Status	Evidence
M-01	Money modeled as <code>double</code> in financial calculations	OPEN	<code>Booking.java:10</code> <code>private final double pricePerNight</code> unchanged
M-02	No Spring Profiles — verbose SQL logging unconditional	OPEN	<code>application.properties:7,10-11</code> : <code>show-sql=true</code> , <code>SQL=DEBUG</code> , <code>BasicBinder=TRACE</code> . No <code>application-dev.properties</code> exists.
M-03	<code>ddl-auto=update</code> with no migration tool; <code>Esquema_BD.sql</code> orphaned	OPEN	<code>application.properties:8</code> unchanged; no Flyway/Liquibase in <code>pom.xml</code>
M-04	Dockerfile declares <code>EXPOSE 8081</code> ; application listens on 8082	OPEN	<code>Dockerfile:4</code> still <code>EXPOSE 8081</code> ; <code>application.properties:16</code> still <code>server.port=8082</code>
M-05	Frontend API URL relies on relative path with no proxy configured	PARTIAL	<code>api.ts</code> introduced with <code>baseUrl</code> : <code>import.meta.env.VITE_API_URL</code> ; <code>.env</code> sets absolute URL. Immediate 404 resolved. No <code>server.proxy</code> ; <code>.env</code> tracked in git (N-02).
M-06	No router-level navigation guards	OPEN	<code>router/index.js</code> has no <code>beforeEach</code> hook
M-07	<code>getUsers()</code> loads entire table with no pagination	OPEN	<code>UserService.java:49</code> : <code>userRepository.findAll().stream()...</code>
M-08	No application-level logging in service classes	OPEN	<code>UserService.java</code> has no <code>Logger</code> or <code>@Slf4j</code>
M-09	REST HTTP status codes wrong on mutation endpoints	OPEN	<code>createUser()</code> → 200 (should be 201); <code>deleteUser()</code> → 200 (should be 204)

2.4 Low Severity

ID	Title	Status
L-01	Scaffold controllers (<code>TestController</code> , <code>HelloController</code>) deployed in production source	OPEN
L-02	<code>HelloWorld.vue</code> and unused assets (<code>vite.svg</code> , <code>vue.svg</code> , <code>hero.png</code> , <code>style.css</code>) still present	OPEN
L-03	Empty <code>common/</code> and <code>security/</code> packages — scaffolded, never filled (0 files each, confirmed)	OPEN
L-04	Orphaned <code>package-lock.json</code> at workspace root (<code>NovaFacts/package-lock.json</code>)	OPEN
L-05	Stale <code>backend/</code> directory at workspace root (only <code>.idea/</code> and <code>.gitignore</code>)	OPEN
L-06	"Forgot password" is a dead <code></code> — no implementation exists (<code>LoginView.vue:68</code>)	OPEN
L-07	<code>authService.ts:16</code> email validation: <code>!email.includes('@')</code> — dangerously weak	OPEN
L-08	"Remember me" stores only the email in <code>localStorage</code> — <code>user_remembered</code> is never read back	OPEN
L-09	Frontend is a mixed JS/TS codebase with no <code>tsconfig.json</code> at project root — TS migration is incomplete	OPEN
L-10	<code>pom.xml</code> metadata fields are all empty stubs (<code><name/></code> , <code><description/></code> , <code><url/></code> , licenses, developers, scm)	OPEN
L-11	<code>BackendApplicationTests.contextLoads()</code> is an empty no-op — always passes, tests nothing	OPEN
L-12	<code>skill_front.md</code> documentation references deleted files and contradicts current code	OPEN

3. New Findings (Third Pass)

N-01 HIGH NEW `createUser()` hardcodes `roleId = 1` with no validation

Category Backend / Data Integrity

Files `auth/service/UserService.java:40-42`

```
user.setNombre(request.getUsername());  
user.setRoleId(1); // hardcoded – always Administrador  
user.setActivo(true);
```

`roleId` is a NOT NULL foreign key to `rol.id`. Hardcoding it to `1` means every user created via `POST /api/users` is silently forced into the **Administrador** role. The `CreateUserRequest` DTO has no `roleId` field, so there is no mechanism for callers to request any other role.

Additionally: if the `rol` table does not contain a row with `id = 1` (e.g., a fresh environment before seed data is applied), the insert fails with a FK violation propagated as an unhandled 500 (compounded by H-03).

RESOLUTION

Add a `roleId` (and `nombre`) field to `CreateUserRequest`. Validate that the referenced role exists before persisting. Return `400` or `404` with a clear message if it does not.

N-02 MEDIUM NEW Frontend `.env` file with localhost URL tracked by git

Category Security / Configuration

Files `project-frontend/frontend/.env`

```
VITE_API_URL=http://localhost:8082
```

`git ls-files frontend/.env` confirms the file is tracked by the repository. The frontend `.gitignore` matches `*.local` but not `.env`, so `.env` is committed.

Vite's documented convention for untracked local values is `.env.local` (excluded by the existing `*.local` rule). While this specific value is low-sensitivity, committing `.env` establishes a pattern: if a future developer adds a real API URL, an analytics key, or any other value, it enters version control automatically. This `.env` also bakes a `localhost` URL into git history, producing a wrong base URL in any staging or production build if not overridden explicitly.

RESOLUTION

Rename `frontend/.env` to `frontend/.env.local` (already in `.gitignore`). Commit a `frontend/.env.example` with `VITE_API_URL=http://localhost:8082` as documentation. Add `.env` to the frontend `.gitignore` explicitly.

N-03

MEDIUM

NEW

Dual Axios interceptors installed on separate instances

Category Frontend / Architecture

Files `src/services/authService.ts:7-13` · `src/services/api.ts:9-17``authService.ts` installs a Bearer token interceptor on the **global** `axios` object:

```
// authService.ts:7
axios.interceptors.request.use((config) => {
  const token = localStorage.getItem(TOKEN_KEY);
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
```

`api.ts` creates a **separate** Axios instance and installs a second identical interceptor on it:

```
// api.ts:9
api.interceptors.request.use((config) => {
  const token = localStorage.getItem(TOKEN_KEY);
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
```

`authService.ts` then imports and uses the `api` instance. The global `axios` interceptor is a leftover from the pre-`api.ts` implementation that was not removed when `api.ts` was introduced. Any code that imports raw `axios` directly — including third-party library calls or future services — will unexpectedly have a `Bearer` header injected into requests to any domain.

RESOLUTION

Remove the `axios.interceptors.request.use(...)` block from `authService.ts` (lines 7–13). The canonical interceptor is the one on the `api` instance in `api.ts`. Ensure all HTTP calls use the `api` instance.

N-04 **MEDIUM** **NEW** `User.username` Java field maps to `email` DB column — semantic mismatch

Category Backend / Maintainability

Files `auth/entity/User.java:15` · `auth/dto/UserResponse.java`

```
@Column(name = "email", nullable = false, unique = true, length = 150)
private String username; // Java field named 'username' stores an email address
```

The JPA mapping fix correctly routes `username` to the `email` column via `@Column(name = "email")`, and `UserRepository.findByUsername()` generates `WHERE email = ?` correctly. However, the divergence between the Java field name and the database column name creates a persistent maintainability trap: any developer reading `user.getUsername()` or `UserResponse.username` reasonably concludes they are working with a username identifier, not an email address. The two-argument constructor `new User(String username, String password)` compounds the confusion at call sites.

RESOLUTION

Rename the Java field from `username` to `email`. Update `UserRepository` to `findByEmail()`. Update the Spring Security `loadUserByUsername()` implementation to call `findByEmail()` internally while keeping the interface method name for framework compatibility. Update all callers. Zero schema change — the DB column name `email` stays untouched.

N-05 **LOW** **NEW** `createUser()` sets `nombre` to the user's email address

Category Backend / Data Integrity

Files `auth/service/UserService.java:40`

`user.setNombre(request.getUsername())` sets the user's display name to their email address. `nombre` in `usuario` is a human-readable display name field (e.g., "Administrador", "Juan Pérez"), not the login identifier. Every user created via the API will appear in any display context with their email address as their name.

RESOLUTION

Add a `nombre` field to `CreateUserRequest`. Set `user.setNombre(request.getNombre())`. Validate `@NotBlank @Size(max=100)` once H-04 is addressed.

N-06 **LOW** **NEW** **TOKEN_KEY** constant duplicated in two service modules

Category Frontend / Code Quality

Files `src/services/api.ts:3` · `src/services/authService.ts:5`

Both files independently declare `const TOKEN_KEY = 'auth_token'`. If the storage key name changes, it must be updated in two places. A desynchronization would cause `api.ts` to read from a different `localStorage` key than `authService.ts` writes to, silently breaking all authenticated requests.

RESOLUTION

Export `TOKEN_KEY` from `authService.ts` and import it in `api.ts`, or extract it to a shared `constants.ts` module.

4. Architectural Integrity Review

Controller → Service → Repository pattern

The pattern is correctly observed in `auth/`. `AuthController` and `UserController` are thin — they delegate immediately to `UserService` with no business logic. `UserService` is the sole class that touches `UserRepository`. `JwtAuthenticationFilter` correctly calls the `UserDetailsService` Spring Security interface, not `UserRepository` directly. No violations found.

Regressions introduced since v2

Item	Type	Location
Dual axios interceptors (N-03)	Architecture regression	<code>authService.ts:7-13</code> , <code>api.ts:9-17</code>
<code>roleId = 1</code> hardcoded (N-01)	Data integrity regression	<code>UserService.java:41</code>
<code>nombre = email</code> (N-05)	Data integrity regression	<code>UserService.java:40</code>
<code>.env</code> tracked by git (N-02)	Security regression	<code>frontend/.env</code>

5. Updated Action Plan

- 1

Close the UX gap & fix the interceptor regression

1–2 hours · Resolves: C-01, N-03
- 1a. Wire `router.push()` on login success (`LoginView.vue`) — import `useRouter`, replace `console.log` with `router.push('/dashboard')`. The JWT stack is complete; this is the only missing line.
 - 1b. Remove the global `axios` interceptor (`authService.ts:7-13`) — delete the `axios.interceptors.request.use(...)` block. The `api.ts` instance interceptor is the canonical one.

2

Eliminate new regressions

2–4 hours · Resolves: N-01, N-02, N-04, N-05, N-06

- **2a. Fix `roleId` hardcoding** — add `roleId` and `nombre` to `CreateUserRequest`; validate role existence before insert; return 400/404 on invalid role.
- **2b. Rename `User.username` to `User.email`** — update entity, repository (`findByEmail()`), all callers. Zero DB schema impact.
- **2c. Move `frontend/.env` to `.env.local`** — commit `.env.example`; add `.env` to frontend `.gitignore`.
- **2d. Extract `TOKEN_KEY` to a single shared location; import it in `api.ts`.**

3

Security and robustness hardening

1–2 days · Resolves: H-01, H-02, H-03, H-04, H-08, M-07, M-09

- **3a. Add `@ControllerAdvice GlobalExceptionHandler`** — map `DataIntegrityViolationException` → 409, `IllegalArgumentException` → 400, others → 500 with no internal detail.
- **3b. Unify wrong-password error** — change `UserService.java:65` to `ResponseStatusException(UNAUTHORIZED, "Credenciales inválidas")`.
- **3c. Add request validation** — add `spring-boot-starter-validation`, annotate DTOs, add `@Valid` to controller parameters.
- **3d. Delete dead config classes** — remove `com.novafacts.SecurityConfig` and `com.novafacts.config.PasswordConfig`.
- **3e. Fix HTTP status codes** — `@ResponseStatus(HttpStatus.CREATED)` on `createUser()`, `@ResponseStatus(HttpStatus.NO_CONTENT)` on `deleteUser()`.
- **3f. Add pagination** — replace `findAll()` with `findAll(Pageable)` in `getUsers()`; add `page` / `size` query params to controller.

4

Dead code and infrastructure cleanup

Half day · Resolves: H-06, L-01, L-02, M-02, M-04

- **4a. Delete `SecretView.vue`, `SecretDisplay.vue`**; remove `/secret` route from `router/index.js`.
- **4b. Delete `TestController.java`, `HelloController.java`**; delete `HelloWorld.vue` and unreferenced assets.
- **4c. Fix `Dockerfile:4`** — change `EXPOSE 8081` to `EXPOSE 8082`.
- **4d. Create `application-dev.properties` with verbose SQL logging; remove `show-sql`, `hibernate.SQL=DEBUG`, `BasicBinder=TRACE` from base `application.properties`.**
- **4e. Add `tsconfig.json` to `project-frontend/frontend/`** to formalize the TS migration.

5

Frontend route protection and logging

Half day · Resolves: M-06, M-08

- **5a.** Add `router.beforeEach()` keyed on token presence in `localStorage`; apply `meta: { requiresAuth: true }` to all protected routes.
- **5b.** Wire `logout()` from `authService.ts` to a logout button in the UI; clear token, redirect to `/login`.
- **5c.** Add `@Slf4j` to `UserService`; log `INFO` for domain events (user created, login succeeded/failed), `WARN` / `ERROR` for unexpected conditions.

6

Schema migration and core domain implementation

Multi-week · Resolves: C-04, M-01, M-03

- **6a. Add Flyway** — convert `Esquema_BD.sql` to `V1__initial_schema.sql`; switch `ddl-auto` to `validate`. Do this *before* adding any new entity.
- **6b. Implement domain entities** — `Reserva`, `Anticipo`, `Factura`, `Penalidad`, `NotaCredito`, `Devolucion` following the feature-package pattern and Spanish column-naming convention from `Esquema_BD.sql`.
- **6c. Replace `double` with `BigDecimal`** — throughout `Booking`, `BookingValidator`, `InvoiceCalculator`; use `RoundingMode.HALF_UP` at each calculation step.

6. Consolidated Open Issue Register

ID	Severity	Title	Phase
C-04	CRITICAL	Core domain has no API, entity, or persistence layer	6
N-01	HIGH NEW	<code>createUser()</code> hardcodes <code>roleId = 1</code> with no validation	2
H-01	HIGH	Login messages still leak user existence (partial fix)	3
H-02	HIGH	Dead config classes outside component-scan boundary	3
H-03	HIGH	No global exception handler — domain errors surface as 500s	3
H-04	HIGH	No request validation anywhere in the API	3
H-05	HIGH	<code>LoginView.vue</code> completes login but performs no navigation	1
H-06	HIGH	<code>SecretView.vue</code> , <code>SecretDisplay.vue</code> , and <code>/secret</code> route orphaned	4
N-02	MEDIUM NEW	Frontend <code>.env</code> tracked by git	2
N-03	MEDIUM NEW	Dual Axios interceptors installed on separate instances	1
N-04	MEDIUM NEW	<code>User.username</code> maps to <code>email</code> column — semantic mismatch	2
H-08	MEDIUM	Uniqueness partial: no service-level 409 check before DB constraint	3
M-01	MEDIUM	Money modeled as <code>double</code> in financial calculations	6
M-02	MEDIUM	Verbose SQL logging unconditional — no Spring Profiles	4
M-03	MEDIUM	<code>ddl-auto=update</code> with no migration tool; <code>Esquema_BD.sql</code> orphaned	6
M-04	MEDIUM	Dockerfile declares <code>EXPOSE 8081</code> ; application listens on 8082	4
M-05	MEDIUM	Frontend API URL partial — <code>.env</code> tracked in git	2
M-06	MEDIUM	No router-level navigation guards	5
M-07	MEDIUM	<code>getUsers()</code> loads entire table with no pagination	3
M-08	MEDIUM	No application-level logging in service classes	5
M-09	MEDIUM	REST HTTP status codes wrong on mutation endpoints	3
N-05	LOW NEW	<code>nombre</code> set to email address on user creation	2

ID	Severity	Title		Phase
N-06	LOW NEW	TOKEN_KEY	constant duplicated in two modules	2
L-01-L-12	LOW	Scaffold code, dead views, empty packages, stale docs, pom stubs		4